

VeroFlow — Complete Project Summary (Current State)

WHAT VEROFLOW IS

An AI-powered platform that automatically tests Android apps. Instead of writing code, you type an instruction in plain English (e.g., "tap the login button"), and the system:

1. Understands what you want
 2. Looks at the app's current screen
 3. Remembers if it has seen this screen before
 4. Generates working test code automatically
 5. Runs it on a real Android emulator
 6. Shows you Pass/Fail with screenshots
-

PART 1: WHAT'S ACTUALLY WORKING RIGHT NOW

✓ Component 1: Natural Language Understanding (Phase 1)

What it does: Takes your plain English instruction and breaks it into a structured list of actions.

Example:

```
Input: "tap the login button"
Output: [{"action": "tap", "target": "login button"}]
```

How it works: Sends your sentence to a local AI model (via Ollama) with instructions to extract actions in JSON format.

File: `pipeline/phase1_nlp_parser.py`

✓ Component 2: Screenshot Capture (Phase 2)

What it does: Takes a screenshot of whatever is currently showing on the Android emulator.

How it works: Uses Appium's built-in `get_screenshot_as_file()` function, saves the image with a timestamp.

File: `pipeline/phase2_adb_capture.py` (logic now also lives inline in `test_executor.py`)

✓ Component 3: Screen Understanding via Vision AI (Phase 3)

What it does: When a screen is seen for the FIRST time, sends the screenshot to a vision AI model and asks "what screen is this and what's on it?"

Example output: "Login Screen – has email field, password field, login button"

How it works: Uses Ollama's vision model (Qwen2.5-VL) which can "read" images.

File: `pipeline/phase3_screen_labeler.py`

✓ Component 4: Image-to-Vector Conversion (Phase 4)

What it does: Converts every screenshot into a list of 512 numbers (a "fingerprint") that represents what the image looks like.

Why: Two screenshots that look similar will have similar number-fingerprints — this lets the system compare screens mathematically instead of just guessing.

How it works: Uses the CLIP AI model (via `open_clip` library).

File: `pipeline/phase4_clip_encoder.py`

✓ Component 5: Screen Memory / RAG (Phase 6)

What it does: This is the "smart" part. Before analyzing a screen with the (slower, heavier) vision AI, it first checks: *"Have I already seen a screen like this before?"*

How it works:

- Stores every screen's fingerprint (from Phase 4) + its label (from Phase 3) in a local database called ChromaDB
- When a new screenshot comes in, compares its fingerprint against everything stored
- If 85%+ similar to something already stored → **reuses the old label** (fast, no AI call needed)
- If nothing similar found → **labels it fresh** with Phase 3, then stores it for next time

Why this matters: The first time VeroFlow sees your login screen, it takes longer (has to actually analyze it). The second time it sees that same login screen, it's instant — recognized from memory.

File: `pipeline/phase6_chromadb_rag.py`

Note on Phase 5 (filtering): Originally planned as a separate step, but ChromaDB's built-in similarity search already does this exact job — deciding "is this a duplicate or new screen?" No separate file needed; one component absorbed the other's responsibility.

✓ Component 6: Test Code Generation

What it does: Combines everything above — the parsed actions (Phase 1) + the screen label (Phase 3/6) + the actual screenshot — and asks the vision AI to write real, executable Appium test code.

Example output:

```
python
```

```
driver.find_element(AppiumBy.XPATH, "//android.widget.Button[@text='Login']").click
```

File: `pipeline/test_generator.py`

✓ Component 7: Test Execution

What it does: Actually runs the AI-generated code on the Android emulator, using Appium to control it — taps buttons, types text, whatever the code says.

How it works:

1. Installs your uploaded APK on the emulator
2. Screenshots BEFORE running the test
3. Runs Phase 4 + 6 (check memory) → Phase 3 if needed (label screen)
4. Generates code
5. Executes the code
6. Screenshots AFTER
7. Returns Pass/Fail + everything above as one result

File: `pipeline/test_executor.py` (this is the "conductor" that calls all the other pieces in order)

✓ Component 8: Backend API

What it does: A web server that the dashboard talks to — receives APK uploads, receives instructions, triggers the test executor, sends results back.

Key endpoints:

- `POST /upload-apk` — saves an uploaded APK file
- `POST /run-test` — runs the full pipeline (all components above) and returns results
- `/screenshots/...` — serves screenshot images so the dashboard can display them

File: `main.py`

✓ **Component 9: Dashboard (Frontend)**

What it does: The actual webpage where you upload an APK, type an instruction, click "Run Test", and see the result — including the parsed actions, screen recognition status, generated code, and before/after screenshots.

Technology: Next.js (React)

File: `veroflow-dashboard/app/page.js`

PART 2: HOW ALL THE PIECES CONNECT (Full Picture)

DASHBOARD (Next.js) – you upload APK, type instruction

sends to backend

BACKEND (FastAPI) – receives request, saves APK

calls

TEST EXECUTOR – the conductor, runs everything in order

1. Phase 1: "tap login" → structured action list

2. Install APK + take screenshot

3. Phase 4: screenshot → 512-number fingerprint

4. Phase 6: check ChromaDB memory

→ seen before? reuse label

→ new? call Phase 3 to label it, then store it

5. Generate Appium code (actions + label + screenshot)

6. Execute the code on the emulator

7. Screenshot again, package up the full result

returns result

DASHBOARD shows: Pass/Fail, screen label, whether it was reused or new, the generated code, before/after screenshots

PART 3: WHAT'S NOT BUILT YET (Be Upfront About This)

Missing Piece	What It Would Do
Phase 8 — Validation/Retry	If the AI generates broken code, automatically re-ask it (up to 3 tries) instead of just failing
Phase 9 — SSIM Verification	Visually confirm the RIGHT screen was reached after the test, not just "no error occurred"
Self-Healing (MD5 Watcher)	Detect when a new APK version changes the UI, and re-check stored screen memory accordingly
Jenkins CI/CD	Automatically run tests when new code is pushed (planned last, per your team's decision)
Web/Playwright testing	Testing web apps, not just Android (deferred from original proposal scope)

PART 4: STEP-BY-STEP — HOW TO RUN EVERYTHING

Prerequisites (One-Time Setup, Already Done)

- Java, Python, Node.js, Appium, Android Studio, Ollama, Docker all installed
- `qwen2.5vl:3b` model pulled via Ollama
- Python packages installed (`pip install -r requirements.txt`)
- Next.js dashboard created (`veroflow-dashboard` folder)

Every Time You Want to Run/Test

Terminal 1 — Start the Emulator

- Open Android Studio → Virtual Device Manager → click  on your emulator
- Wait until you see the actual Android home screen (fully booted, not just loading)

Terminal 2 — Start Appium

```
bash
appium
```

Leave this running. Don't close this terminal.

Terminal 3 — Start the Backend

```
bash  
  
cd backend  
source venv/bin/activate  
uvicorn main:app --reload
```

Leave this running too.

Terminal 4 — Start the Dashboard

```
bash  
  
cd veroflow-dashboard  
npm run dev
```

Verify Emulator Is Connected (open a 5th terminal briefly)

```
bash  
  
adb devices
```

Should show something like:

```
List of devices attached  
emulator-5554    device
```

If this list is empty, wait longer for the emulator to finish booting.

Now Actually Test It

1. Open your browser: `http://localhost:3000`
2. **Upload an APK** — pick something simple with standard buttons (not a game or Flutter app)
3. **Type an instruction** — start simple: `tap the login button`
4. Click **"Run Test"**
5. Wait 15-40 seconds (first run is slower)
6. Read the result:
 - Status: PASSED or FAILED
 - Screen Recognition box: shows the screen label + whether it was new or reused

- Generated Code: the actual Appium code the AI wrote
- Before/After screenshots

The Important Second Test (Proves Memory Works)

Run the **same instruction again** (or a different one, same screen). Check the "Screen Recognition" box:

- **First run:** "New screen — labeled and stored"
- **Second run:** "Reused from memory (similarity: 0.9x)"

This difference is your actual proof that Phase 4 + 6 (CLIP + ChromaDB) are working — not just installed, but genuinely remembering screens between test runs.

PART 5: IF SOMETHING GOES WRONG

Problem	Likely Cause	Fix
<code>adb devices</code> shows empty	Emulator not fully booted	Wait longer, check Android Studio window
Appium can't connect	Appium not running, or wrong device name	Confirm Terminal 2 shows Appium running; check <code>device_name</code> matches <code>adb devices</code> output
<code>ModuleNotFoundError</code>	Python packages missing or venv not activated	Run <code>source venv/bin/activate</code> then <code>pip install -r requirements.txt</code> again
Dashboard shows nothing / network error	Backend not running, or CORS issue	Confirm Terminal 3 (<code>uvicorn</code>) is running and shows no errors
Screenshots don't display in dashboard	Static file mount missing	Confirm <code>main.py</code> has the <code>/screenshots</code> mount line
Test always says "New screen" even on 2nd run	ChromaDB threshold too strict, or screen genuinely changed slightly	Try the exact same instruction/screen twice in a row without navigating elsewhere first
Generated code fails / wrong element	AI guessed wrong locator (expected at this stage — no validation retry loop yet)	Try a simpler instruction, or a more standard-looking app

ONE-LINE SUMMARY FOR YOUR PROFESSOR

"We have a working core pipeline: natural language gets parsed into structured actions, screenshots get converted into searchable fingerprints, a memory system (ChromaDB) recognizes previously-seen screens instead of re-analyzing them every time, and the AI generates and executes real Appium test code on a live Android emulator — with a working dashboard to run and view all of this. Validation retry logic, visual verification, self-healing, and CI/CD integration are the next planned pieces."